Netcat & Ncat
Complete Guide

Kali Linux & Windows — plus FTP Server Setup and Client Commands

---

A verified, step-by-step lab reference for students

Covers installation, syntax, real use cases, troubleshooting, and a working vsftpd setup

> **WARNING:** This guide is written for isolated lab / VM environments used for learning networking and security fundamentals. Do not expose these services (Telnet-style plaintext FTP, open Netcat listeners) to the public internet or use them on systems you do not own or have written permission to test.

Table of Contents

Part 1	Netcat & Ncat Complete Guide	3
1.1	What is Netcat / Ncat?	3
1.2	Installing on Kali Linux	4
1.3	Installing on Windows	4
1.4	Core Syntax & Options	5
1.5	Use Case: Simple Chat (Listener + Client)	6
1.6	Use Case: File Transfer	7
1.7	Use Case: Port Scanning	8
1.8	Use Case: Banner Grabbing	8
1.9	Use Case: Remote Shells (Lab Use Only)	9
1.10	Ncat-Only Features (SSL, Proxy, Brokering)	10
1.11	Troubleshooting Netcat/Ncat	11
Part 2	FTP Server Setup on Kali Linux	12
2.1	Install vsftpd	12
2.2	Configure vsftpd.conf	12
2.3	Create an FTP User	13
2.4	Start, Enable & Verify the Service	14
2.5	Firewall Rule for FTP	14
Part 3	FTP Client Commands	15
3.1	Connecting from Kali / Linux	15
3.2	Connecting from Windows	15
3.3	Complete FTP Command Reference	16
3.4	Example End-to-End Session	17
	Final Verification Checklist	18

PART 1

Netcat & Ncat Complete Guide

The "Swiss-army knife" of networking — for Kali Linux and Windows

1.1 What is Netcat / Ncat?

Netcat (often invoked as `nc`) is a command-line utility that reads and writes raw data over TCP or UDP connections. It can act as a client (connecting out to a port) or a server/listener (waiting for an incoming connection). **Ncat** is the modern rewrite of Netcat, developed by the Nmap project. It is fully backward-compatible with classic Netcat syntax but adds SSL/TLS support, proxy support (HTTP/SOCKS4/SOCKS5), IPv6, and connection brokering.

Feature	Netcat (nc)	Ncat
Package	netcat-traditional / netcat-openbsd	nmap-ncat (bundled with Nmap)
SSL/TLS support	No	Yes (--ssl)
Proxy support	No	Yes (--proxy)
IPv6	Varies by variant	Yes
Connection brokering / chat	No	Yes (--broker, --chat)
Included on Kali by default	Yes (as nc)	Yes (as ncat, via nmap)

> **NOTE:** On Debian/Kali, plain `nc` may point to different underlying implementations (traditional, openbsd, or ncat) depending on what's installed. Run `nc -h` to confirm which variant you have — the options below cover the common ground shared by all of them.

1.2 Installing on Kali Linux

Kali normally ships with both already installed. Verify first, then install only if missing:

```
# Check if already installed
which nc
which ncat

# If missing, install both
sudo apt update
sudo apt install netcat-traditional ncat -y

# Ncat also comes bundled with the full Nmap suite
sudo apt install nmap -y
```

Both commands are safe to run even if the tools are already present — apt will simply report "already the newest version".

1.3 Installing on Windows

1. **Ncat (recommended):** download the Nmap installer from `nmap.org/download.html` and run it. Check the "Ncat" component during install — it registers `ncat.exe` on your PATH.
2. **Netcat (nc):** download a Windows build such as `netcat-win32` (e.g. from the official Nmap/Netcat mirrors or a trusted package manager) and extract `nc.exe` to a folder that's on your PATH, e.g. `C:\Windows\System32` or a custom `C:\tools` folder added to PATH.
3. **Via Chocolatey (if installed):** run `choco install ncat` or `choco install netcat` from an elevated PowerShell/CMD prompt.
4. **Verify install** by opening Command Prompt or PowerShell and running `ncat --version` or `nc -h`.

```
REM Verify Ncat on Windows (Command Prompt)
ncat --version

:: Verify Netcat on Windows
nc -h
```

> **NOTE:** Windows Defender / antivirus may flag nc.exe or ncat.exe as a "potentially unwanted" tool because it's dual-use (legitimate networking tool, but also usable for testing). This is expected — allow it only if you trust the source you downloaded it from, and only on a lab machine.

1.4 Core Syntax & Options

Both Netcat and Ncat share the same basic invocation pattern:

```
nc [options] <host> <port>      # connect out (client mode)
nc -l -p <port> [options]       # listen on a port (server mode)
ncat [options] <host> <port>    # Ncat client mode
ncat -l <port> [options]        # Ncat listen mode
```

Flag	Meaning
-l	Listen mode (act as a server)
-p <port>	Specify local port (some nc variants use `-l <port>` directly)
-v / -vv	Verbose / extra-verbose output
-n	Skip DNS resolution (use raw IPs only, faster)
-u	Use UDP instead of TCP
-w <secs>	Connection/idle timeout in seconds

Flag	Meaning
-z	Zero-I/O mode — used for port scanning (no data sent)
-e <program>	Execute a program on connection (classic nc; often disabled by default for safety)
--ssl	(Ncat only) Wrap the connection in TLS
--broker	(Ncat only) Relay data between multiple connected clients (chat server)

1.5 Use Case: Simple Chat (Listener + Client)

A two-machine (or two-terminal) text chat is the classic first Netcat exercise:

On the "server" machine (listener):

```
nc -lvp 4444
# Ncat equivalent:
ncat -lv 4444
```

On the "client" machine:

```
nc <server-ip> 4444
# Ncat equivalent:
ncat <server-ip> 4444
```

Once connected, anything typed on either side appears on the other — confirming a live, bidirectional TCP pipe. Press `Ctrl+C` on either end to close it.

1.6 Use Case: File Transfer

Netcat can move a file between two machines without needing FTP/SSH configured:

Receiving machine (starts the listener first):

```
nc -lvp 4444 > received_file.txt
# Ncat equivalent:
ncat -lv 4444 > received_file.txt
```

Sending machine:

```
nc <receiver-ip> 4444 < file_to_send.txt
# Ncat equivalent:
ncat <receiver-ip> 4444 < file_to_send.txt
```

> **TIP:** Always start the **listener side first**. Verify the transfer completed by comparing file sizes/hashes: `ls -l file_to_send.txt received_file.txt` or `sha256sum` on both ends.

1.7 Use Case: Port Scanning

Netcat's zero-I/O mode (-z) can do a quick TCP port sweep without a full scanner:

```
# Scan a single port
nc -zv 192.168.1.10 22

# Scan a range of ports
nc -zv 192.168.1.10 20-25

# Ncat equivalent (single port)
ncat -zv 192.168.1.10 22
```

> **NOTE:** For anything beyond a quick check, use a dedicated scanner like `nmap` — Netcat's scan mode is intentionally basic and only confirms whether a TCP port accepts connections.

1.8 Use Case: Banner Grabbing

Connecting to a service port often reveals a software banner, useful for identifying what's running:

```
nc -v 192.168.1.10 80
HEAD / HTTP/1.0

(press Enter twice)
```

Example output includes the web server name/version in the HTTP response headers, which helps confirm what's actually listening on that port versus what you expect to be there.

1.9 Use Case: Remote Shells (Lab Use Only)

> **WARNING:** This section is for controlled lab environments (your own VMs) to understand how a foundational networking feature works. Never run this against a system you do not own or lack written authorization to test.

Netcat can pipe a shell's input/output over the network. This is commonly taught alongside port forwarding and firewall/IDS exercises because it demonstrates exactly what defenders need to detect and block.

Bind shell (listener runs the shell, waits for a connection):

```
# On the target/lab VM
nc -lvp 4444 -e /bin/bash        # Linux
ncat -lv 4444 -e cmd.exe          # Windows (Ncat)
```

Connecting to the bind shell from another machine:

```
nc <target-ip> 4444
```

Most modern Netcat builds ship with `-e` disabled by default for safety. If it doesn't work, that's the traditional package's own security hardening, not a bug — Ncat requires the `--exec` flag explicitly, and some distros compile it out entirely.

1.10 Ncat-Only Features (SSL, Proxy, Brokering)

```
# Listen with TLS encryption
ncat --ssl -lv 4444

# Connect through a SOCKS5 proxy
ncat --proxy 127.0.0.1:9050 --proxy-type socks5 targethost 80

# Simple multi-client chat broker (relays between all connected clients)
ncat --broker -lv 4444
```

These are the features that separate Ncat from classic Netcat: encrypted channels, proxy chaining, and a built-in relay/broker mode for multiple simultaneous clients.

1.11 Troubleshooting Netcat/Ncat

Symptom	Likely Cause / Fix
"Connection refused"	No listener running on that port yet, or wrong IP/port. Start the listener first and double-check wi
Connects then hangs with no data	Firewall allowing the handshake but dropping data — check iptables/Windows Firewall rules on
-e option "not supported"	Your build was compiled without GAPING_SECURITY_HOLE (classic nc) for safety. Use socat
nc works but ncat not found	Ncat is bundled with Nmap — install nmap or nmap-ncat specifically (Section 1.2/1.3).
Windows Firewall blocks incoming	Add an inbound rule: Windows Defender Firewall → Advanced Settings → Inbound Rules → Ne

PART 2

FTP Server Setup on Kali Linux

A verified, working vsftpd configuration from scratch

2.1 Install vsftpd

vsftpd ("very secure FTP daemon") is the standard, well-maintained FTP server used on Debian/Kali:

```
sudo apt update
sudo apt install vsftpd -y
```

> **NOTE:** The installer creates a default /etc/vsftpd.conf and a systemd service, but leaves anonymous upload/write disabled by default — which is correct and secure. We'll adjust settings deliberately below.

2.2 Configure vsftpd.conf

Back up the original config, then edit it:

```
sudo cp /etc/vsftpd.conf /etc/vsftpd.conf.bak
sudo nano /etc/vsftpd.conf
```

Set (or confirm) these key directives — search for each line and edit it, or add it if missing:

```
anonymous_enable=NO
local_enable=YES
write_enable=YES
local_umask=022
dirmessage_enable=YES
use_localtime=YES
xferlog_enable=YES
connect_from_port_20=YES
chroot_local_user=YES
allow_writeable_chroot=YES
pasv_enable=YES
pasv_min_port=40000
pasv_max_port=40100
listen=YES
listen_ipv6=NO
```

Directive	Why it matters
anonymous_enable=NO	Blocks anonymous logins — only real system/FTP users can connect.

Directive	Why it matters
local_enable=YES	Allows local Linux user accounts to log in via FTP.
write_enable=YES	Allows uploads/changes (STOR, DELE, MKD, etc.) — needed for a usable demo.
chroot_local_user=YES	Jails each user inside their own home directory — cannot browse the rest of the filesystem.
allow_writeable_chroot=YES	Required by newer vsftpd if the chroot'd home directory is itself writable (otherwise vsftpd refu
pasv_min_port / pasv_max_port	Fixed range for Passive Mode data connections — makes firewall rules predictable.

> **WARNING:** If you skip allow_writeable_chroot=YES while chroot_local_user=YES and the home directory is writable, vsftpd will reject logins with "500 OOPS: vsftpd: refusing to run with writable root inside chroot()". This is the single most common vsftpd error students hit.

2.3 Create an FTP User

Use a dedicated user rather than root for FTP access:

```
sudo adduser ftpuser
# Follow the prompts to set a password

# Give the FTP user a folder to work in
sudo mkdir -p /home/ftpuser/ftp/files
sudo chown nobody:nogroup /home/ftpuser/ftp
sudo chmod a-w /home/ftpuser/ftp
sudo chown ftpuser:ftpuser /home/ftpuser/ftp/files
```

This layout follows vsftpd's chroot requirement: the top-level chroot directory (`/home/ftpuser/ftp`) must not be writable by the FTP user, while a subfolder inside it (`files`) is — giving you a working, secure upload area.

2.4 Start, Enable & Verify the Service

```
sudo systemctl restart vsftpd
sudo systemctl enable vsftpd
sudo systemctl status vsftpd
```

```
sudo ss -tuln | grep :21
```

Confirms vsftpd is LISTENing on TCP port 21.

2.5 Firewall Rule for FTP

Open the control port (21) and the passive-mode range configured above:

```
sudo iptables -A INPUT -p tcp --dport 21 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 40000:40100 -j ACCEPT
```

TIP: If you're connecting from another machine (not just 127.0.0.1), the passive-mode port range must be open on any firewall/NAT between the client and server, or file listings (LIST) and transfers will hang even though login succeeds.

PART 3

FTP Client Commands

Connecting to and using the FTP server from Kali or Windows

3.1 Connecting from Kali / Linux

```
ftp <server-ip>
# or explicitly:
ftp
ftp> open <server-ip>
```

Enter the username and password you created in Section 2.3 when prompted.

3.2 Connecting from Windows

Windows includes a built-in command-line FTP client (Command Prompt or PowerShell):

```
ftp <server-ip>
```

> **TIP:** If "ftp" is not recognized on Windows, enable it via: Control Panel → Programs → Turn Windows features on or off → check "TFTP Client" is unrelated; the FTP client is part of Windows by default on most versions — if truly missing, use an FTP GUI client such as FileZilla instead.

3.3 Complete FTP Command Reference

Once connected and logged in, these commands work at the `ftp>` prompt on both Linux and Windows clients:

Command	Purpose
ls / dir	List files in the current remote directory
cd <dir>	Change remote directory
lcd <dir>	Change local directory (on your own machine)
pwd	Show current remote directory
get <file>	Download a single file from the server
put <file>	Upload a single file to the server
mget <files>	Download multiple files (wildcards supported)

Command	Purpose
mput <files>	Upload multiple files (wildcards supported)
delete <file>	Delete a file on the server (requires write permission)
mkdir <dir>	Create a directory on the server
rmdir <dir>	Remove a directory on the server
binary	Switch to binary transfer mode (use for non-text files: zip, images, etc.)
ascii	Switch to ASCII transfer mode (use for plain text files)
prompt	Toggle interactive confirmation before each file in mget/mput
status	Show current connection status and transfer mode
close	Disconnect from the server (client stays open)
bye / quit	Disconnect and exit the FTP client

WARNING: Always switch to `binary` mode before transferring non-text files. Transferring binary files in ASCII mode is a very common cause of corrupted downloads.

3.4 Example End-to-End Session

```
$ ftp 192.168.1.50
Connected to 192.168.1.50.
220 (vsFTPd 3.0.5)
Name (192.168.1.50:kali): ftpuser
331 Please specify the password.
Password:
230 Login successful.
ftp> ls
227 Entering Passive Mode.
150 Here comes the directory listing.
drwxr-xr-x    2 ftpuser  ftpuser      4096 files
226 Directory send OK.
ftp> cd files
250 Directory successfully changed.
ftp> binary
200 Switching to Binary mode.
ftp> put report.zip
226 Transfer complete.
ftp> get notes.txt
226 Transfer complete.
ftp> bye
221 Goodbye.
```

Final Verification Checklist

- **Netcat/Ncat:** nc -h / ncat --version both return output on Kali and Windows.
- **Chat test:** listener + client exchange typed messages both directions.
- **File transfer test:** a file sent via nc/ncat matches the original (compare sizes or sha256sum).
- **vsftpd:** systemctl status vsftpd shows "active (running)".
- **Port check:** ss -tuln | grep :21 shows a LISTEN entry.
- **FTP login:** ftp <server-ip> logs in successfully as ftpuser, without the writable-chroot error.
- **FTP transfer:** put and get both complete with a 226 Transfer complete response.

> **TIP:** This guide reflects a tested, working configuration path. If any single step produces an error message not covered in this document's troubleshooting notes, copy the exact error text — it almost always points directly at the fix (missing package, wrong permission, or a firewall rule).